



Orientações para resolução da lista

- Resolva os exercícios utilizando exclusivamente os conceitos abordados em aula até o momento (variáveis, expressões, estruturas condicionais e de repetição).
- Antes de implementar, leia atentamente o enunciado e procure identificar claramente:
 - quais são as entradas do problema;
 - qual processamento deve ser realizado;
 - qual saída é esperada.
- Sempre que possível, elabore casos de teste antes da implementação. Teste situações típicas, limites e casos extremos.
- Evite começar diretamente pelo código. Pense na lógica da solução e, se necessário, descreva os passos em linguagem natural ou pseudocódigo.
- Utilize nomes de variáveis significativos, que representem claramente o papel de cada valor no problema.
- Em exercícios com entrada contínua, tenha atenção especial às condições de parada (sentinelas), como valores que indicam o fim da leitura.
- Em estruturas de repetição, verifique cuidadosamente:
 - a condição de parada;
 - a atualização das variáveis de controle;
 - possíveis situações de loop infinito.
- Em problemas que envolvem decisões, analise se as condições são independentes ou se devem ser mutuamente exclusivas.
- Sempre valide seu programa com diferentes conjuntos de entrada para garantir que o comportamento está correto.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver sua autonomia na resolução de problemas.

1. Peça um número inteiro positivo ao usuário e determine se ele é primo utilizando `while`.
2. Peça uma base e um expoente inteiro não negativo ao usuário e calcule a potência manualmente, sem usar operador de exponenciação, utilizando um `for`.
3. Peça ao usuário um número binário (por exemplo: 1101), e converta-o para decimal utilizando um `while`.
4. Peça um valor inteiro positivo ao usuário e determine quantas cédulas de R\$100, R\$50, R\$20, R\$10, R\$5 e R\$1 são necessárias para formar esse valor, utilizando estruturas de repetição.
5. Faça um programa de um jogo de adivinhação. O programa deve gerar um número aleatório entre 1 e 10. O usuário deve informar palpites, e o programa deve indicar se o número correto é maior ou menor. O jogo continua até que o usuário acerte.
6. Crie um jogo em que o usuário escolhe par ou ímpar, informa um número e joga contra o computador (que gera um número aleatório). O programa deve mostrar o resultado da soma, identificar se é par ou ímpar e informar o vencedor. O jogo deve se repetir até o usuário desejar sair.
7. Considere a série:

$$S = \frac{2}{1^2} + \frac{2 \cdot 3}{2^2} + \frac{2 \cdot 3 \cdot 4}{3^2} + \frac{2 \cdot 3 \cdot 4 \cdot 5}{4^2} + \dots$$

Escreva um programa que leia um valor inteiro n e calcule os n primeiros termos dessa série e sua soma final.

Observação: o numerador de cada termo é formado por um produtório sequencial começando em 2, e o denominador é o quadrado da posição do termo.

8. Considere a série:

$$S = \frac{1}{2 \cdot 1} + \frac{1 \cdot 3}{2^2 \cdot 2} + \frac{1 \cdot 3 \cdot 5}{2^3 \cdot 3} + \frac{1 \cdot 3 \cdot 5 \cdot 7}{2^4 \cdot 4} + \dots$$

Escreva um programa que leia um valor inteiro n e calcule os n primeiros termos dessa série e sua soma final.

9. Considere a série:

$$S = \frac{1 \cdot 1}{1!} + \frac{(1+2) \cdot (1 \cdot 2)}{2!} + \frac{(1+2+3) \cdot (1 \cdot 2 \cdot 3)}{3!} + \frac{(1+2+3+4) \cdot (1 \cdot 2 \cdot 3 \cdot 4)}{4!} + \dots$$

Escreva um programa que leia um valor inteiro n e calcule os n primeiros termos dessa série e sua soma final.

- 10.** Peça ao usuário que informe uma sequência de números inteiros. A leitura deve ser encerrada quando o valor 0 for digitado (o valor 0 não deve ser considerado na sequência).

Um trecho da sequência é considerado crescente consecutivo quando cada número é estritamente maior que o anterior. Desenvolva um programa que determine o maior tamanho de um trecho crescente consecutivo presente na sequência informada.

Exemplo:

Entrada: 3, 5, 7, 2, 4, 6, 8, 1, 0

Saída: 4

(Referente ao trecho: 2, 4, 6, 8)

- 11.** Peça ao usuário que informe uma sequência de números inteiros. A leitura deve ser encerrada quando o valor 0 for digitado (o valor 0 não deve ser considerado na sequência).

Um padrão é identificado quando ocorre uma sequência de pelo menos três números consecutivos iguais.

Desenvolva um programa que verifique se existe esse padrão na sequência informada.

Exemplos:

Entrada: 1, 2, 2, 2, 3, 0

Saída: Existe padrão

Entrada: 1, 2, 2, 3, 0

Saída: Não existe padrão

- 12.** Peça ao usuário um número inteiro positivo N.

Desenvolva um programa que imprima uma sequência de linhas formando uma escada numérica crescente, iniciando em 1 e indo até N, de forma que cada linha contenha os números de 1 até o valor da linha.

Em seguida, o programa deve imprimir a escada inversa, reduzindo progressivamente até 1.

Exemplo (N = 4):

```
1 1
2 12
3 123
4 1234
5 123
6 12
7 1
```

- 13.** Fernanda, Francisca e Flávia são três irmãs. Sabe-se que Fernanda nasceu antes de Francisca e Flávia nasceu depois de Francisca. Faça um programa que leia

três números inteiros correspondentes às idades das irmãs (em qualquer ordem) e imprima a idade de Francisca.

14. Você está desenvolvendo um módulo de RH para uma empresa de tecnologia. O sistema deve gerenciar a escala de trabalho de uma equipe de 7 engenheiros. Cada engenheiro tem um ID de 1 a 7. Para distribuir os dias de plantão, a empresa decidiu que engenheiros com ID par não podem fazer plantão em dias ímpares do mês, e engenheiros com ID ímpar não podem fazer plantão em dias pares.

Escreva um programa que receba o dia do mês (1 a 30) e, em seguida, peça ao usuário para digitar o ID do engenheiro que deseja escalar. O programa deve validar se a escala é permitida segundo a regra acima e exibir "Escala Confirmada" ou "Funcionário fora da escala".

15. Uma provedora de nuvem cobra por hora de uso de instâncias de servidor. As regras de faturamento são:

- As primeiras 100 horas custam R\$ 5,00/hora.
- De 101 a 500 horas, o custo cai para R\$ 4,00/hora para as horas excedentes.
- Acima de 500 horas, o custo cai para R\$ 2,50/hora para as horas excedentes.

O usuário deve inserir a quantidade de horas consumidas por um cliente. O programa deve calcular o valor total da fatura.

16. Uma cooperativa de agricultores possui um silo central com capacidade limitada de quilos de sementes para distribuir entre seus associados. Você deve criar um programa que registre as retiradas de sementes até que o estoque se esgote ou até que não haja sementes suficientes para atender a um pedido específico. O programa deve solicitar ao usuário a capacidade máxima do silo e o peso desejado por cada agricultor da fila. Se o valor solicitado for maior do que o que resta no silo, o sistema deve exibir uma mensagem informando que o pedido não pode ser atendido por falta de estoque e encerrar as operações do dia. Caso contrário, ele subtrai o valor do total e continua pedindo o próximo pedido. Ao final, o programa deve exibir o total de quilos distribuídos e quantos agricultores conseguiram retirar sementes com sucesso.

17. Analise o trecho de código abaixo, que simula o cálculo de bônus de produtividade de uma pequena equipe de 4 funcionários, e responda:

- Qual será o valor final impresso das variáveis `soma_bonus` e `total_passos`?
- Como a estrutura condicional impacta o resultado final da variável `soma_bonus`?

```
1 soma_bonus = 0
2 total_passos = 0
3 limite_produtividade = 50
4
5 for i in range(4):
6     print("Analisando funcionario...")
```

```

7     pontos = (i + 1) * 20
8     total_passos = total_passos + 1
9
10    if pontos > limite_produtividade:
11        soma_bonus = soma_bonus + 100
12    else:
13        soma_bonus = soma_bonus + 10

```

18. Um sistema deve classificar o embarque de passageiros com base em prioridade, classe (econômica ou executiva), categoria (Comum, Prata, Ouro) e condição de atraso.

As regras de embarque devem seguir uma hierarquia clara de prioridade. Passageiros que estão atrasados sempre embarcam por último, independentemente de qualquer outra condição. Entre os demais, aqueles que possuem prioridade (como idosos, gestantes ou pessoas com necessidades especiais) embarcam primeiro. Em seguida, embarcam os passageiros da classe executiva. Depois disso, são chamados os passageiros da classe econômica, respeitando a ordem de suas categorias de fidelidade: primeiro os da categoria Ouro, depois os da categoria Prata e, por fim, os da categoria Comum.

Faça um programa que leia essas informações e determine o grupo de embarque do passageiro.

19. Uma universidade está desenvolvendo um sistema para controlar o acesso de estudantes a um laboratório especializado. Para cada estudante, o sistema deve receber a idade, se possui treinamento prévio para uso do laboratório e se apresentou autorização formal do professor responsável. O processo de entrada deve continuar até que seja informada uma idade igual a 0, indicando o encerramento da triagem. Idades negativas devem ser consideradas inválidas e, nesse caso, o registro deve ser desconsiderado.

As regras de liberação são as seguintes: estudantes com 18 anos ou mais podem entrar no laboratório se tiverem realizado o treinamento. Estudantes com menos de 18 anos somente podem entrar se tiverem realizado o treinamento e também apresentarem autorização formal. Caso um estudante não atenda aos requisitos, o sistema deve informar que o acesso foi negado. Ao final do processamento, o sistema deve informar quantos estudantes tiveram acesso liberado, quantos tiveram acesso negado e qual foi o percentual de liberações entre os registros válidos. Caso nenhum registro válido tenha sido processado, o sistema deve informar que não há dados suficientes para análise.

Desenvolva um algoritmo que resolva esse problema e forneça sua implementação em Python.

20. Uma pequena distribuidora deseja monitorar a situação do estoque de seus produtos ao final de um inventário. Para cada produto, o sistema deve receber o nome do item, a quantidade atualmente disponível em estoque e a quantidade mínima ideal para funcionamento normal da empresa. O processo deve continuar até que o usuário informe a palavra “fim” como nome do produto. Quantidades negativas

devem ser consideradas inválidas, e qualquer produto com dado inválido deve ser completamente ignorado.

Para cada produto válido, o sistema deve classificar sua situação da seguinte forma: “estoque crítico” quando a quantidade disponível for menor que 50% da quantidade mínima ideal, “estoque baixo” quando a quantidade disponível for menor que a quantidade mínima ideal, mas ainda não estiver em situação crítica, e “estoque adequado” quando a quantidade disponível for maior ou igual à quantidade mínima. Além de exibir a classificação de cada produto, o sistema deve, ao final, informar quantos produtos ficaram em situação crítica, quantos ficaram com estoque baixo e quantos estavam adequados. Também deve informar o nome do produto com maior diferença positiva entre estoque disponível e estoque mínimo, considerando apenas produtos válidos. Caso nenhum produto válido tenha sido processado, o sistema deve informar essa situação explicitamente.

Desenvolva um algoritmo que resolva esse problema e forneça sua implementação em Python.

- 21.** Uma empresa de logística deseja criar um sistema para analisar o desempenho diário de seus entregadores. Para cada entrega realizada, o sistema deve receber o tempo previsto para conclusão e o tempo efetivamente gasto, ambos em minutos. O processo deve continuar até que o tempo previsto informado seja igual a 0. Tempos negativos ou tempos efetivos iguais a 0 devem ser considerados inválidos e devem ser desconsiderados do processamento.

Uma entrega deve ser classificada como “adiantada” quando o tempo efetivo for menor que o previsto, “no prazo” quando os dois tempos forem iguais, e “atrasada” quando o tempo efetivo for maior que o previsto. Ao final do processamento, o sistema deve informar a quantidade de entregas adiantadas, no prazo e atrasadas, além do atraso médio entre as entregas classificadas como atrasadas. Caso não exista nenhuma entrega atrasada, o sistema deve informar que não há atraso médio a calcular. Como armadilha adicional, observe que uma entrega muito adiantada não deve influenciar o cálculo do atraso médio, pois esse cálculo deve considerar apenas os casos em que houve atraso real.

Desenvolva um algoritmo que resolva esse problema e forneça sua implementação em Python.

- 22.** Uma empresa de segurança digital está testando um mecanismo simples de validação de senhas criadas por seus usuários. O sistema deve receber sucessivas senhas digitadas, encerrando a execução quando o usuário informar a palavra “encerrar”. Cada senha deve ser analisada individualmente. Uma senha será considerada válida apenas se possuir pelo menos 8 caracteres, contiver pelo menos um número e não contiver espaços em branco. Senhas vazias ou compostas apenas por espaços devem ser tratadas como inválidas.

Para cada senha informada, o sistema deve indicar se ela é válida ou inválida. Ao final, o sistema deve informar quantas senhas válidas foram registradas, quantas senhas inválidas foram registradas e qual foi a maior senha válida digitada, em termos de número de caracteres. Caso nenhuma senha válida tenha sido informada, o sistema deve deixar isso explícito. O desafio desse problema está em perceber

que a validação não depende apenas do tamanho da string, mas da análise de suas características internas, exigindo uma varredura caractere a caractere.

Desenvolva um algoritmo que resolva esse problema e forneça sua implementação em Python.

23. Considere o código a seguir:

```
1 n = 6
2 i = 0
3
4 while i < n:
5     if i % 2 == 0:
6         n = n - 1
7     i += 1
8
9 print(i, n)
```

Determine os valores finais de `i` e `n` e explique como a modificação dinâmica de `n` influencia o número de iterações do loop.

24. Considere o código a seguir:

```
1 x = 0
2 y = 10
3
4 if x != 0 and y / x > 1:
5     print("A")
6 else:
7     print("B")
```

Determine se ocorre erro em tempo de execução, qual é a saída do programa e explique o papel da avaliação de curto-circuito.

25. Considere o código a seguir:

```
1 i = 1
2 total = 0
3
4 while i <= 5:
5     total += i
6     if total > 6:
7         i += 2
8     else:
9         i += 1
10
11 print(total)
```

Determine o valor final de `total`, descreva a sequência de valores assumidos por `i` e discuta se o algoritmo sempre termina.

26. Considere o código a seguir:

```
1 maior = 0
2 n = int(input())
3
4 while n != 0:
5     if n > maior:
6         maior = n
7     n = int(input())
8
9 print(maior)
```

Determine se o algoritmo está correto para todos os casos, apresente um contraexemplo, corrija o algoritmo e explique por que a versão corrigida é adequada.

27. Projete um algoritmo que leia números inteiros positivos, rejeite valores inválidos, pare quando o usuário digitar zero e informe quantidade, média, maior e menor valor. Descreva invariantes do algoritmo, o fluxo lógico da solução, identifique casos críticos e implemente.

28. Considere o código a seguir:

```
1 contador = 0
2 soma = 0
3
4 for i in range(5):
5     n = int(input())
6
7     if n >= 0:
8         soma += n
9         contador += 1
10
11 media = soma / contador
12 print(media)
```

Análise se o programa pode falhar, em quais condições isso ocorre, classifique o tipo de erro e proponha uma correção.

29. Você está programando a lógica de segurança de um drone de entregas. Enquanto o equipamento está em trânsito para entrega, o sistema receberá todos os dados do usuário e exibirá as mensagens de alerta na tela de saída. O drone só tem permissão para decolar se passar por uma sequência de verificações de sensores que consomem bateria. O drone inicia com uma carga de bateria informada pelo operador. O processo de decolagem consiste em 5 etapas de checagem. Em cada etapa, o programa gera um consumo fixo de 1,7% da bateria total inicial. Além disso, em cada etapa, o programa recebe do sensor de vento a velocidade do vento no momento atual (em km/h). Se, em qualquer uma das 5 etapas, a velocidade do vento for superior a 30 km/h, o drone deve cancelar a decolagem imediatamente por segurança. Caso a bateria caia abaixo de 10% durante qualquer momento das checagens, o processo

também deve ser interrompido com o alerta "Bateria Insuficiente". O programa deve informar ao final se a decolagem foi "Autorizada", "Cancelada por Vento Forte" ou "Abortada por Bateria", indicando em qual etapa o problema ocorreu e o nível final de carga.

- 30. (Desafio)** Você foi contratado para ajudar uma startup que está desenvolvendo um novo aplicativo chamado SafeLogin, usado por estudantes para acessar materiais da faculdade. Nos primeiros testes, a equipe percebeu que muitos usuários estavam criando senhas muito fracas, o que representa um risco de segurança. Por isso, pediram sua ajuda para criar uma verificação automática de senha. Durante o cadastro, o sistema pede ao usuário que crie uma senha. Antes de aceitar, o sistema precisa verificar se essa senha é segura o suficiente. Uma senha será considerada válida apenas se atender às seguintes regras:

- Deve ter pelo menos 8 caracteres
- Deve conter pelo menos um número
- Deve conter pelo menos uma letra

Caso a senha não atenda a qualquer uma dessas regras, o sistema deve informar que a senha é inválida. O seu código deve conter apenas estruturas condicionais, de repetição, variáveis e operadores.

Dica: Pesquise por novas estratégias no python que possam te ajudar a resolver essa questão.

Questões de múltipla escolha:

1. Em sistemas computacionais que operam sob restrições de desempenho, é comum o uso de mecanismos adaptativos que ajustam dinamicamente limites de execução com base no comportamento observado durante o processamento. Esse tipo de estratégia é utilizado, por exemplo, em algoritmos de controle de carga, nos quais o número de iterações pode ser reduzido ao longo da execução para evitar sobrecarga ou garantir responsividade.

Considere que uma equipe de desenvolvimento implementou o seguinte algoritmo em Python para simular esse comportamento adaptativo:

```
1 n = 5
2 i = 0
3 soma = 0
4
5 while i < n:
6     soma += i
7     if soma > 4:
8         n -= 1
9     i += 1
```

Com base na análise do funcionamento desse algoritmo, avalie as assertivas a seguir:

I. O laço pode ser encerrado antes de atingir o número de iterações originalmente definido por n , em função da modificação dinâmica dessa variável.

II. O valor final de $soma$ é necessariamente igual à soma dos inteiros de 0 até 4, independentemente das alterações realizadas em n durante a execução.

III. A alteração da variável n ao longo da execução pode antecipar a condição de parada do laço, reduzindo a quantidade total de iterações realizadas.

É correto o que se afirma em:

- A) Apenas I e II.
 - B) Apenas I e III.
 - C) Apenas II e III.
 - D) Apenas I.
 - E) I, II e III.
2. Em sistemas críticos, especialmente aqueles que lidam com operações sensíveis, é fundamental garantir que determinadas expressões sejam avaliadas de forma segura, evitando erros em tempo de execução. Linguagens como Python utilizam estratégias de avaliação condicional que podem interromper a análise de expressões lógicas

quando o resultado já pode ser determinado, contribuindo para a robustez do sistema.

Considere o seguinte trecho de código:

```
1 x = 0
2 y = 10
3
4 if x != 0 and y / x > 1:
5     resultado = "OK"
6 else:
7     resultado = "ERRO"
```

Com base na análise do comportamento do programa, avalie as assertivas a seguir:

I. A expressão de divisão presente na condição não é executada durante a avaliação do comando condicional.

II. A execução do programa não resulta em erro em tempo de execução.

III. O valor atribuído à variável `resultado` ao final da execução é "OK".

É correto o que se afirma em:

- A) Apenas I e II.
- B) Apenas II e III.
- C) Apenas I.
- D) Apenas III.
- E) I, II e III.

3. Em sistemas de autenticação, expressões lógicas são frequentemente utilizadas para combinar múltiplos critérios de acesso, como validação de usuário, senha e privilégios administrativos. A correta interpretação dessas expressões é fundamental para garantir que o controle de acesso funcione conforme o esperado.

Considere o seguinte trecho de código em Python:

```
1 usuario = True
2 senha = False
3 admin = True
4
5 if usuario and senha or admin:
6     acesso = True
7 else:
8     acesso = False
```

Com base na análise do comportamento do código apresentado, assinale a alternativa correta:

- A) A expressão lógica é equivalente a `(usuario and senha) or admin`.
 - B) O acesso será negado ao final da execução.
 - C) A expressão lógica é ambígua, podendo gerar interpretações distintas.
 - D) O resultado da expressão depende da ordem de avaliação das operações lógicas.
 - E) O código apresenta erro de execução.
4. Em linguagens de programação, a estrutura de decisão pode ser organizada de diferentes formas, como encadeamentos condicionais ou estruturas independentes. A escolha inadequada dessas estruturas pode levar a comportamentos inesperados.

Considere o seguinte trecho de código em Python:

```
1 n = 70
2
3 if n >= 60:
4     print("A")
5 if n >= 40:
6     print("B")
7 else:
8     print("C")
```

Com base na análise do comportamento desse código, avalie as assertivas a seguir:

I. Não há exclusão mútua entre as condições avaliadas no código.

II. Para $n = 70$, duas saídas são exibidas durante a execução do programa.

III. Substituir o segundo `if` por `elif` altera o comportamento do programa, promovendo exclusão mútua entre as condições.

É correto o que se afirma em:

- A) Apenas I e II.
- B) Apenas I e III.
- C) Apenas II e III.
- D) I, II e III.
- E) Apenas I.

5. Um sistema de controle de acesso é utilizado para autorizar a entrada de visitantes em um evento. De acordo com as regras estabelecidas, o acesso é permitido nas seguintes condições:
- pessoas com 18 anos ou mais têm acesso liberado;
 - pessoas com 16 ou 17 anos podem entrar apenas se estiverem acompanhadas de autorização.

Para implementar essa lógica, foi desenvolvido o seguinte trecho de código em Python:

```
1 idade = 17
2 autorizacao = True
3
4 if idade >= 18 or idade >= 16 and autorizacao:
5     permitido = True
6 else:
7     permitido = False
```

Com base na análise do comportamento do código apresentado, assinale a alternativa correta:

- A) A expressão lógica é equivalente a `(idade >= 18 or idade >= 16) and autorizacao`.
 - B) O acesso será negado ao final da execução.
 - C) O acesso será permitido ao final da execução.
 - D) O código apresenta erro lógico, pois não respeita as regras descritas.
 - E) A expressão lógica é inválida em Python.
6. Em sistemas computacionais, é comum a necessidade de validar múltiplas condições simultaneamente, especialmente em processos de verificação de integridade ou consistência de dados. Em alguns casos, deseja-se executar uma ação quando pelo menos uma dessas condições não é satisfeita.

Considere o seguinte trecho de código em Python:

```
1 if not (a and b):
2     print("OK")
```

Com base na equivalência entre expressões lógicas, assinale a alternativa que apresenta uma forma equivalente à condição utilizada no código:

- A) `not a and not b`
- B) `not a or not b`

- C) a or b
- D) a and not b
- E) not a or b

7. Em um sistema de análise de desempenho, uma métrica acumulada é calculada a partir de uma sequência de valores inteiros consecutivos. Para cada valor processado, o sistema aplica a seguinte regra: valores pares contribuem positivamente para a métrica, enquanto valores ímpares representam penalidades, sendo subtraídos do total.

Considere o seguinte trecho de código em Python, no qual o valor de entrada N define o limite superior do processamento:

```
1 total = 0
2
3 for i in range(1, N + 1):
4     if i % 2 == 0:
5         total += i
6     else:
7         total -= i
```

Analise as afirmações a seguir e a relação entre elas:

- I. Para qualquer valor inteiro positivo de N , o valor final de `total` será negativo quando N for ímpar.

PORQUE

- II. O algoritmo alterna sistematicamente entre operações de subtração e soma ao longo das iterações, iniciando com uma subtração.

Assinale a alternativa correta:

- A) As afirmações I e II são verdadeiras, e II justifica corretamente I.
 - B) As afirmações I e II são verdadeiras, mas II não justifica I.
 - C) A afirmação I é verdadeira, e II é falsa.
 - D) A afirmação I é falsa, e II é verdadeira.
 - E) As afirmações I e II são falsas.
8. Em um sistema de processamento incremental, uma variável acumuladora é atualizada ao longo de um laço de repetição, enquanto o avanço do índice de controle

depende do estado parcial da própria execução. Esse tipo de estratégia pode ser encontrado em algoritmos adaptativos, nos quais a progressão do processamento não ocorre de maneira fixa, mas varia conforme condições avaliadas durante o percurso.

Considere o seguinte trecho de código em Python:

```
1 i = 0
2 soma = 0
3
4 while i < 5:
5     soma += i
6     if soma > 3:
7         i += 2
8     else:
9         i += 1
```

Analise as afirmações a seguir e a relação entre elas:

I. O valor da variável *i* cresce de forma não uniforme ao longo da execução.

PORQUE

II. O incremento aplicado à variável *i* depende do valor assumido pela variável *soma* em cada iteração.

Assinale a alternativa correta:

- A) As afirmações I e II são verdadeiras, e II justifica corretamente I.
 - B) As afirmações I e II são verdadeiras, mas II não justifica I.
 - C) A afirmação I é verdadeira, e II é falsa.
 - D) A afirmação I é falsa, e II é verdadeira.
 - E) As afirmações I e II são falsas.
9. Em sistemas de processamento de dados, é comum a necessidade de identificar valores extremos (como o menor ou o maior) a partir de uma sequência de entradas fornecidas pelo usuário. A correta inicialização das variáveis é um aspecto fundamental nesses algoritmos, pois pode influenciar diretamente o resultado final.

Considere o seguinte trecho de código em Python, utilizado para determinar o menor valor entre três números informados pelo usuário:

```
1 menor = 0
2
3 for i in range(3):
4     n = int(input())
5     if n < menor:
6         menor = n
```

Com base na análise do comportamento desse algoritmo, avalie as assertivas a seguir:

I. O algoritmo pode produzir resultados incorretos, dependendo dos valores informados.

II. O erro ocorre quando todos os valores informados são positivos.

III. O problema está relacionado à forma como a variável `menor` é inicializada.

Assinale a alternativa correta:

A) Apenas I e II estão corretas.

B) Apenas I e III estão corretas.

C) Apenas II e III estão corretas.

D) Todas estão corretas.

E) Apenas I está correta.

10. Em um sistema de validação de dados, uma sequência de números é analisada para identificar quantos valores positivos consecutivos aparecem em cada bloco de entrada. O sistema considera que cada bloco termina quando um valor negativo é informado, e que a sequência completa termina quando o valor 0 é digitado.

Considere o seguinte trecho de código em Python:

```
1 n = int(input())
2
3 while n != 0:
4     contador = 0
5
6     while n > 0:
7         contador += 1
8         n = int(input())
9
10    print(contador)
11
12    n = int(input())
```

Considere os seguintes casos de teste:

Caso 1: Entrada: 1, 2, 3, -1, 0 Saída esperada: 3

Caso 2: Entrada: 5, 6, -2, 7, 8, -1, 0 Saída esperada: 2, 2

Caso 3: Entrada: -3, 4, 5, -1, 0 Saída esperada: 0, 2

Com base na execução do código e nos casos de teste apresentados, conclui-se que o algoritmo:

A) processa corretamente todos os blocos de números positivos, independentemente da posição de valores negativos.

- B) ignora corretamente valores negativos e contabiliza apenas os blocos de números positivos, conforme esperado.
- C) não processa corretamente blocos que começam imediatamente após valores negativos.
- D) apresenta comportamento consistente com os casos de teste, produzindo sempre as saídas esperadas.
- E) garante que todos os blocos de entrada sejam processados sem perda de dados.